

Top 70 JavaScript Interview Questions & Answers (2026)

Frontend Developer — Fresher Edition

Q1. What is JavaScript?

A1. JavaScript is a high-level programming language used to make websites interactive and dynamic. HTML builds the structure, CSS adds style, and JavaScript adds behavior — like button clicks, popups, and form validation. It runs directly in the browser.

```
document.getElementById("btn").onclick = function() {  
    alert("Button Clicked!");  
}
```

Q2. What is the difference between var, let, and const?

A2. All three are used to declare variables in JavaScript but they work differently.

1. var has function scope and we can re-assign and re-declare var variable.
2. let has block scope and we can re-assign let variable but we cannot re-declare let variable.
3. const has block scope and we cannot re-assign and re-declare const variable.

```
var name = "Rahul";  
var name = "Priya"; // OK – var allows re-declaration  
let age = 22;  
age = 23;           // OK – can re-assign  
// let age = 25; // ERROR – cannot re-declare  
const city = "Delhi";  
// city = "Mumbai"; // ERROR – cannot re-assign
```

Q3. What is Hoisting?

A3. Hoisting means JavaScript moves all variable and function declarations to the top before running the code. var is hoisted with value undefined. Functions are fully hoisted so they can be called before writing them. let and const give error if used before declaration.

```
console.log(name); // undefined – var is hoisted  
var name = "Rahul";  
greet(); // Works! – function is fully hoisted  
function greet() {
```

```
    console.log("Hello!");  
  }  
  // console.log(age); // ERROR – let not initialized yet  
  let age = 22;
```

Q4. What are Data Types in JavaScript?

A4. Data types tell us what kind of value a variable holds. There are two categories — Primitive and Non-Primitive. Primitive types are String, Number, Boolean, Undefined, and Null. Non-Primitive types are Object, Array, and Function.

```
let name = "Rahul";           // String  
let age = 22;                 // Number  
let isPass = true;           // Boolean  
let score;                   // Undefined  
let data = null;             // Null  
let person = { name: "Rahul" }; // Object  
let fruits = ["Apple", "Mango"]; // Array
```

Q5. What is the difference between == and ===?

A5. == checks only the value and ignores the type — so it converts types before comparing. === checks both value AND type — no conversion happens. Always use === in real projects because it is more accurate.

```
console.log(5 == "5"); // true – value same, type ignored  
console.log(5 === "5"); // false – type is different  
console.log(0 == false); // true  
console.log(0 === false); // false
```

Q6. What is typeof in JavaScript?

A6. typeof is an operator that returns the data type of a value as a string. It is used to check what type of value a variable holds. One famous bug — typeof null returns "object" which is wrong, but it is how JavaScript works.

```
console.log(typeof "Hello"); // "string"  
console.log(typeof 42); // "number"  
console.log(typeof true); // "boolean"  
console.log(typeof undefined); // "undefined"  
console.log(typeof null); // "object" – JS bug!
```

```
console.log(typeof {});      // "object"
console.log(typeof []);      // "object"
```

Q7. What is the difference between null and undefined?

A7. Both mean empty value but they are used in different situations.

1. undefined means a variable is declared but no value is assigned yet. JavaScript sets it automatically.
2. null means the developer has set the variable to empty on purpose and typeof null returns object which is a known JavaScript bug.

```
let a;
console.log(a); // undefined – no value assigned yet
let b = null;
console.log(b); // null – intentionally set to empty
console.log(undefined == null); // true
console.log(undefined === null); // false
```

Q8. What are Functions in JavaScript?

A8. A function is a reusable block of code that performs a specific task. You write it once and call it as many times as needed. There are 3 main ways — Function Declaration, Function Expression, and Arrow Function.

1. Function Declaration:- Starts with the function keywords

```
function greet(name) {
  return "Hello " + name;
}
console.log(greet("Rahul")); // Hello Rahul
// 2. Function Expression:- Stored in a variable
const add = function(a, b) {
  return a + b;
};
// 3. Arrow Function:- Short & cleaner way to write functions using
// =>.
const multiply = (a, b) => a * b;
console.log(multiply(3, 4)); // 12
```

Q9. What is an Arrow Function?

A9. Arrow function is a shorter and cleaner way to write functions, introduced in ES6. It uses the => symbol. The main difference is that arrow functions do NOT have their own this keyword.

```
// Normal function
function add(a, b) {
  return a + b;
}
// Arrow function – shorter
const add = (a, b) => a + b;
// Single parameter – no brackets needed
const double = n => n * 2;
console.log(double(5)); // 10
// No parameter
const greet = () => "Hello!";
```

Q10. What is Scope in JavaScript?

A10. Scope means where a variable can be accessed in the code. There are two types of scope.

1. Local Scope:- In local scope variable declared inside the function and only accessible inside that function.
2. Global Scope:- In Global scope variable declared outside the function or block and accessible anywhere in page or module.

```
let global = "I am global"; // usable everywhere
function myFunc() {
  let funcVar = "function only";
  console.log(global); // Works
  console.log(funcVar); // Works
}
if (true) {
  let blockVar = "block only";
  console.log(blockVar); // Works inside block
}
// console.log(blockVar); // ERROR – not accessible here
```

Q11. What is Closure in JavaScript?

A11. Closure means inner function can use variables of outer function even after outer function is done.

```
function outer() {
  let count = 0;
```

```
function inner() {
    count++;
    console.log(count);
}
return inner;
}
const counter = outer();
counter(); // 1
counter(); // 2
counter(); // 3
// count is still remembered – this is closure
```

Q12. What is the difference between function declaration and function expression?

A12. Both are used to create functions but they work differently.

1. Function Declaration is hoisted — we can call it before writing it.
2. Function Expression is not hoisted — we must write it before calling it.

```
// Declaration – can call before writing
greet(); // Works!
function greet() {
    console.log("Hello!");
}
// Expression – cannot call before writing
// sayHi(); // ERROR
const sayHi = function() {
    console.log("Hi!");
};
sayHi(); // Works now
```

Q13. What is the this keyword in JavaScript?

A13. this keyword refers to the object that calls the function. In an object method this refers to that object. Arrow functions do not have their own this.

```
const person = {
    name: "Rahul",
    greet: function() {
        console.log("Hello, I am " + this.name);
    }
};
person.greet(); // Hello, I am Rahul
```

Q14. What is an Array in JavaScript?

A14. An array is used to store multiple values in a single variable. Array stores multiple values in order and they are accessed by index number starting from 0.

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
console.log(fruits[1]); // Banana
console.log(fruits.length); // 3
fruits.push("Orange"); // Add at end
fruits.pop();           // Remove from end
```

Q15. What is the difference between map() and forEach()?

A15. Both are used to loop through an array but map() returns a new array with changes and forEach() does not return anything.

```
let numbers = [1, 2, 3];
// map – returns new array
let doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6]
// forEach – no return value
numbers.forEach(n => console.log(n * 2));
// prints 2, 4, 6 but returns undefined
```

Q16. What is the difference between filter() and map()?

A16. Both return a new array but map() transforms every item and returns new array with same length and filter() returns only those items that pass the given condition.

```
let numbers = [1, 2, 3, 4, 5];
// map – transforms every item
let doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
// filter – keeps only matching items
let evens = numbers.filter(n => n % 2 === 0);
console.log(evens); // [2, 4]
```

Q17. What is the reduce() method in JavaScript?

A17. reduce() is used to reduce an entire array into a single value. It is mostly used to calculate sum, product, or total of array items.

```
let numbers = [1, 2, 3, 4, 5];
let sum = numbers.reduce((total, n) => total + n, 0);
console.log(sum); // 15
// Real example – total cart price
let cart = [{ price: 100 }, { price: 200 }, { price: 50 }];
let total = cart.reduce((sum, item) => sum + item.price, 0);
console.log(total); // 350
```

Q18. What is the difference between find() and filter()?

A18. Both are used to search in an array but they work differently. find() returns only the first item that matches the condition and filter() returns all items that match the condition as a new array.

```
let numbers = [1, 2, 3, 4, 5];
// find – returns FIRST match only
let first = numbers.find(n => n > 2);
console.log(first); // 3
// filter – returns ALL matches
let all = numbers.filter(n => n > 2);
console.log(all); // [3, 4, 5]
```

Q19. What is an Object in JavaScript?

A19. An object is used to store data in key and value pairs. Keys are called properties and functions inside an object are called methods.

```
let person = {
  name: "Rahul",
  age: 22,
  city: "Delhi",
  greet: function() {
    console.log("Hello, I am " + this.name);
  }
};
console.log(person.name); // Rahul
console.log(person["age"]); // 22
person.greet(); // Hello, I am Rahul
```

Q20. What is Destructuring in JavaScript?

A20. Destructuring is a short way to extract values from an array or object into separate variables.

```
// Array Destructuring
let [a, b, c] = [10, 20, 30];
console.log(a); // 10
console.log(b); // 20
// Object Destructuring
let person = { name: "Rahul", age: 22 };
let { name, age } = person;
console.log(name); // Rahul
console.log(age); // 22
```

Q21. What is the Spread Operator in JavaScript?

A21. Spread operator (...) is used to expand an array or object into individual elements. It is used to copy, merge arrays and objects.

```
// Copy array
let arr1 = [1, 2, 3];
let arr2 = [...arr1];
console.log(arr2); // [1, 2, 3]
// Merge arrays
let a = [1, 2];
let b = [3, 4];
let merged = [...a, ...b];
console.log(merged); // [1, 2, 3, 4]
// Copy object
let person = { name: "Rahul", age: 22 };
let copy = { ...person, city: "Delhi" };
console.log(copy); // { name: "Rahul", age: 22, city: "Delhi" }
```

Q22. What is the Rest Parameter in JavaScript?

A22. Rest parameter ... collects multiple arguments into a single array inside a function. It looks like spread operator but it is used in function parameters.

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3)); // 6
console.log(sum(1, 2, 3, 4, 5)); // 15
```

Q23. What is the difference between Spread and Rest operator?

A23. Both use (...) but they work differently.

1. Spread operator expands an array or object into individual elements.
2. Rest parameter collects multiple arguments into a single array.

```
// Spread – expands array
let arr = [1, 2, 3];
console.log(...arr); // 1 2 3
// Rest – collects into array
function sum(...numbers) {
  console.log(numbers); // [1, 2, 3]
}
sum(1, 2, 3);
```

Q24. What is Template Literal in JavaScript?

A24. Template literal uses backticks to create strings. We can add variables and expressions inside `\${}`. It also supports multi-line strings easily.

```
let name = "Rahul";
let age = 22;
// Old way
let msg1 = "Hello, my name is " + name;
// Template Literal – cleaner
let msg2 = `Hello, my name is ${name} and I am ${age}`;
console.log(msg2); // Hello, my name is Rahul and I am 22
// Multi-line string
let text = `Line 1
Line 2
Line 3`;
```

Q25. What is a Callback Function in JavaScript?

A25. A callback function is a function passed as an argument to another function and called later. It is mostly used in setTimeout, event listeners, and API calls.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
function done() {
  console.log("Greeting is done!");
}
greet("Rahul", done);
```

```
// Hello Rahul  
// Greeting is done!
```

Q26. What is a Promise in JavaScript?

A26. A Promise is an object that represents a value that will be available in the future, either as success or error. It has 3 states:

1. Pending:- It means waiting.
2. Fulfilled:- It means success.
3. Rejected:- It means failed.

We use `.then()` for success and `.catch()` for error.

```
let myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  if (success) {  
    resolve("Data fetched successfully!");  
  } else {  
    reject("Something went wrong!");  
  }  
});  
myPromise  
  .then(result => console.log(result)) // Success  
  .catch(error => console.log(error)); // Error
```

Q27. What is async and await in JavaScript?

A27. `async` and `await` is a simple and easy way to handle Promises. `async` keyword is used before a function and it always returns a promise. `await` keyword pauses the function until the promise is resolved.

```
async function getUser() {  
  try {  
    let response = await fetch("https://jsonplaceholder.typicode.com/users/1");  
    let data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Error:", error);  
  }  
}  
getUser();
```

Q28. What is the difference between Promise and async await?

A28. Both are used to handle async operations but they work differently.

1. Promise uses .then() and .catch() which make messy code with multiple calls.
2. async await is simple and easy to read — it makes async code look like normal code.

```
// Promise way
fetch("/user")
  .then(res => res.json())
  .then(data => console.log(data));
// async await way – cleaner
async function getData() {
  let res = await fetch("/user");
  let data = await res.json();
  console.log(data);
}
```

Q29. What is the Event Loop in JavaScript?

A29. JavaScript runs one task at a time but Event Loop allows async tasks to run in background without stopping the main code. Sync code runs first then async code runs later.

```
console.log("1 - Start");
setTimeout(() => {
  console.log("2 - setTimeout");
}, 0);
console.log("3 - End");
// Output:
// 1 - Start
// 3 - End
// 2 - setTimeout – runs last even with 0ms delay
```

Q30. What is the difference between synchronous and asynchronous JavaScript?

A30. Both are ways to run code but they work differently.

1. Synchronous code runs line by line — each line waits for the previous one to finish.
2. Asynchronous code does not wait — it runs in background and other code keeps running.

```
// Synchronous – waits for each line
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");
// Output: 1, 2, 3 in order
```

```
// Asynchronous – does not wait
console.log("Step 1");
setTimeout(() => console.log("Step 2"), 1000);
console.log("Step 3");
// Output: Step 1, Step 3, Step 2
```

Q31. What is DOM in JavaScript?

A31. DOM stands for Document Object Model. When browser loads HTML it creates a tree-like structure of all elements. JavaScript uses DOM to read, change, add or remove HTML elements on the page.

```
// Access element
let heading = document.getElementById("title");
// Change content
heading.textContent = "New Title";
// Change style
heading.style.color = "red";
// Change class
heading.classList.add("active");
heading.classList.remove("old");
```

Q32. What is the difference between `getElementById` and `querySelector`?

A32. Both are used to select HTML elements but they work differently.

1. `getElementById` selects element using ID only and it is faster.
2. `querySelector` is more flexible — it can select by CSS selector.

```
// getElementById – only by ID
let e1 = document.getElementById("title");
// querySelector – any CSS selector
let e2 = document.querySelector("#title"); // by ID
let e3 = document.querySelector(".card");  // by class
let e4 = document.querySelector("p");      // by tag
// querySelectorAll – returns ALL matching elements
let allCards = document.querySelectorAll(".card");
```

Q33. What is Event Handling in JavaScript?

A33. Events are user's actions like click, hover, and typing. We use `addEventListener` to listen events and call a function when they happen.

```
let btn = document.getElementById("myBtn");
// Click event
btn.addEventListener("click", function() {
    alert("Button clicked!");
});
// Keyboard event
document.addEventListener("keydown", function(e) {
    console.log("Key pressed:", e.key);
});
// Input event
let input = document.getElementById("search");
input.addEventListener("input", function(e) {
    console.log("Typing:", e.target.value);
});
```

Q34. What is Event Bubbling in JavaScript?

A34. Event Bubbling means when we click on a child element the event starts from that child element and moves up to parent elements. By default all events bubble up in JavaScript.

```
document.getElementById("parent").addEventListener("click", () => {
    console.log("Parent clicked");
});
document.getElementById("child").addEventListener("click", () => {
    console.log("Child clicked");
});
// When child button is clicked output:
// Child clicked
// Parent clicked – event bubbled up to parent
```

Q35. What is event.preventDefault() in JavaScript?

A35. preventDefault() stops the default browser behavior of an element. For example it stops form default behavior like form submission or link navigation to another page.

```
// Prevent form submission
let form = document.getElementById("myForm");
form.addEventListener("submit", function(e) {
    e.preventDefault();
    console.log("Form submitted without page reload!");
});
// Prevent link navigation
let link = document.getElementById("myLink");
```

```
link.addEventListener("click", function(e) {
  e.preventDefault();
  console.log("Link click prevented!");
});
```

Q36. What is localStorage in JavaScript?

A36. localStorage is used to store data in the browser. Data stored in localStorage stays even after the browser is closed. We use setItem to save, getItem to get, and removeItem to delete data.

```
// Save data
localStorage.setItem("username", "Rahul");
// Get data
let name = localStorage.getItem("username");
console.log(name); // Rahul
// Remove data
localStorage.removeItem("username");
// Store object
let user = { name: "Rahul", age: 22 };
localStorage.setItem("user", JSON.stringify(user));
let saved = JSON.parse(localStorage.getItem("user"));
```

Q37. What is the difference between localStorage and sessionStorage?

A37. Both store data in the browser but they work differently.

1. localStorage stores data even after browser is closed — it is permanent.
2. sessionStorage deletes data when the tab is closed — it is temporary.

```
// localStorage – stays forever
localStorage.setItem("username", "Rahul");
// sessionStorage – deleted on tab close
sessionStorage.setItem("token", "abc123");
```

Q38. What is JSON in JavaScript?

A38. JSON stands for JavaScript Object Notation. It is a format used to store and transfer data. JSON.stringify() converts object to JSON string and JSON.parse() converts JSON string to object.

```
let person = { name: "Rahul", age: 22 };
// Object to JSON string
let jsonString = JSON.stringify(person);
console.log(jsonString);
```

```
// '{"name":"Rahul","age":22}'  
// JSON string to Object  
let backToObject = JSON.parse(jsonString);  
console.log(backToObject.name); // Rahul
```

Q39. What is the Fetch API in JavaScript?

A39. fetch() is used to make HTTP requests and get data from a server or API. It returns a Promise. It is the modern way to call REST APIs in JavaScript.

```
async function getUser() {  
  try {  
    let res = await fetch("https://jsonplaceholder.typicode.com/users/  
1");  
    let data = await res.json();  
    console.log(data);  
  } catch(err) {  
    console.log("Error:", err);  
  }  
}  
getUser();
```

Q40. What is try catch in JavaScript?

A40. try catch is used to handle errors in JavaScript. We write our main code inside try block. If any error comes in try block then catch block handles it and finally block runs every time.

```
try {  
  let data = JSON.parse("invalid json");  
  console.log(data);  
} catch(error) {  
  console.log("Error:", error.message);  
} finally {  
  console.log("This always runs!");  
}
```

Q41. What is the difference between call(), apply(), and bind()?

A41. All three are used to set this manually but they work differently.

1. call() calls the function immediately and passes arguments one by one.
2. apply() calls the function immediately and passes arguments as an array.
3. bind() does not call the function immediately — it returns a new function.

```
function greet(city, country) {
  console.log(this.name + " from " + city + ", " + country);
}
let person = { name: "Rahul" };
// call – arguments one by one
greet.call(person, "Delhi", "India");
// Rahul from Delhi, India
// apply – arguments as array
greet.apply(person, ["Delhi", "India"]);
// Rahul from Delhi, India
// bind – returns new function
let boundGreet = greet.bind(person, "Delhi", "India");
boundGreet(); // Rahul from Delhi, India
```

Q42. What is a Higher Order Function in JavaScript?

A42. A Higher Order Function is a function that takes another function as an argument or returns a function. `map()`, `filter()`, and `reduce()` are all higher order functions.

```
// Takes function as argument
function applyOperation(numbers, operation) {
  return numbers.map(operation);
}
let nums = [1, 2, 3, 4];
let doubled = applyOperation(nums, n => n * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Q43. What is the difference between `for...in` and `for...of`?

A43. Both are used to loop but they work differently.

1. `for...in` loops work through keys and mostly used for object.
2. `for...of` loops work through values and mostly used for array or string.

```
// for...in – loops over KEYS
let person = { name: "Rahul", age: 22 };
for (let key in person) {
  console.log(key); // name, age
}
// for...of – loops over VALUES
let fruits = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
```

```
console.log(fruit); // Apple, Banana, Mango
}
```

Q44. What is the difference between shallow copy and deep copy?

A44. Both are used to copy objects but they work differently.

1. Shallow copy copies only top level properties — nested objects are still shared.
2. Deep copy copies everything completely — nothing is shared.

```
let original = { name: "Rahul", address: { city: "Delhi" } };
// Shallow copy – nested object is shared
let shallow = { ...original };
shallow.address.city = "Mumbai";
console.log(original.address.city); // "Mumbai" – original changed!
// Deep copy – completely independent
let deep = JSON.parse(JSON.stringify(original));
deep.address.city = "Agra";
console.log(original.address.city); // "Mumbai" – original not changed
```

Q45. What is Debouncing in JavaScript?

A45. Debouncing is a technique used to control how many times a function is called. In Debouncing function runs after the user stops an action.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
let searchAPI = debounce((query) => {
  console.log("Searching for:", query);
}, 500);
input.addEventListener("input", () => searchAPI(input.value));
// API called only after user stops typing for 500ms
```

Q46. What is Throttling in JavaScript?

A46. Throttling is a technique used to control how many times a function is called. But in Throttling function runs once in a fixed time interval. It is mostly used in scroll and resize events.

```
function throttle(fn, limit) {
  let lastCall = 0;
  return function(...args) {
    let now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      fn(...args);
    }
  };
}

let handleScroll = throttle(() => {
  console.log("Scroll event fired!");
}, 1000); // runs only once per second
window.addEventListener("scroll", handleScroll);
```

Q47. What is IIFE in JavaScript?

A47. IIFE stands for Immediately Invoked Function Expression. It is a function that runs immediately after it is created. It is used to create a private scope and avoid global variables.

```
(function() {
  let message = "I run immediately!";
  console.log(message);
})();
// Arrow IIFE
(() => {
  console.log("Arrow IIFE!");
})();
// message is not accessible outside
// console.log(message); // ERROR
```

Q48. What is the difference between break and continue in JavaScript?

A48. Both are used inside loops but they work differently.

1. break completely stops the loop and exits it.
2. continue skips the current step and moves to the next one.

```
// break – stops loop completely
for (let i = 0; i < 5; i++) {
  if (i === 3) break;
  console.log(i); // 0, 1, 2
}
// continue – skips current iteration
```

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) continue;  
  console.log(i); // 0, 1, 2, 4  
}
```

Q49. What is Optional Chaining in JavaScript?

A49. Optional chaining ?. is used to safely access nested object properties. If the property does not exist it returns undefined instead of throwing an error.

```
let user = {  
  name: "Rahul",  
  address: {  
    city: "Delhi"  
  }  
};  
console.log(user.address?.city); // "Delhi"  
console.log(user.phone?.number); // undefined – no error!  
// Very useful with API data  
let data = null;  
console.log(data?.user?.name); // undefined – no error!
```

Q50. What is the Nullish Coalescing Operator in JavaScript?

A50. Nullish coalescing operator ??(double question mark) returns the right side value only when left side is null or undefined. It is different from ||(double pipe operator) which checks all falsy values including 0 and "" (empty string).

```
let name = null;  
console.log(name ?? "Guest"); // "Guest"  
let count = 0;  
console.log(count || 10); // 10 – WRONG! 0 is valid value  
console.log(count ?? 10); // 0 – CORRECT! only checks null/undefined  
let text = "";  
console.log(text ?? "default"); // "" – not replaced
```

Q51. What is the difference between undefined and not defined in JavaScript?

A51. Both mean no value but they are different.

1. undefined means variable is declared but no value is assigned yet.
2. not defined means variable is never declared — it gives a ReferenceError.

```
let a;  
console.log(a); // undefined – declared but no value  
// console.log(b); // ReferenceError – b is not defined at all
```

Q52. What is NaN in JavaScript?

A52. NaN stands for Not a Number. It is returned when a math operation fails or when we try to do math with a non-numeric value. We use isNaN() to check if a value is NaN.

```
console.log(10 / "hello"); // NaN  
console.log("abc" * 2);    // NaN  
console.log(isNaN("hello")); // true  
console.log(isNaN(42));    // false  
// NaN is not equal to itself  
console.log(NaN === NaN); // false
```

Q53. What is the difference between slice() and splice()?

A53. Both are used on arrays but they work differently.

1. slice() returns a part of array without changing the original array.
2. splice() adds or removes items from array and changes the original array.

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];  
// slice – does not change original  
let sliced = fruits.slice(1, 3);  
console.log(sliced); // ["Banana", "Mango"]  
console.log(fruits); // original unchanged  
// splice – changes original array  
fruits.splice(1, 2);  
console.log(fruits); // ["Apple", "Orange"]
```

Q54. What is the difference between push(), pop(), shift(), and unshift()?

A54. All four are array methods but they work differently.

1. push() adds item at the end of array.
2. pop() removes item from the end of array.
3. unshift() adds item at the start of array.
4. shift() removes item from the start of array.

```
let arr = ["Banana", "Mango"];  
arr.push("Orange"); // ["Banana", "Mango", "Orange"]  
arr.pop();           // ["Banana", "Mango"]
```

```
arr.unshift("Apple"); // ["Apple", "Banana", "Mango"]
arr.shift();          // ["Banana", "Mango"]
```

Q55. What is the difference between `indexOf()` and `includes()`?

A55. Both are used to search in an array but they work differently.

1. `indexOf()` returns the index number of the item. If item is not found it returns -1.
2. `includes()` returns true or false — whether item exists or not.

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits.indexOf("Banana")); // 1
console.log(fruits.indexOf("Grapes")); // -1 – not found
console.log(fruits.includes("Mango")); // true
console.log(fruits.includes("Grapes")); // false
```

Q56. What is Object Destructuring in JavaScript?

A56. Object destructuring is a short way to extract values from an object into separate variables. We use `{}` curly brackets to destructure an object.

```
let person = { name: "Rahul", age: 22, city: "Delhi" };
// Without destructuring
let name = person.name;
let age = person.age;
// With destructuring – shorter
let { name, age, city } = person;
console.log(name); // Rahul
console.log(age);  // 22
// With default value
let { country = "India" } = person;
console.log(country); // India
```

Q57. What is the difference between `Object.keys()`, `Object.values()`, and `Object.entries()`?

A57. All three are used to get data from an object but they return different things.

1. `Object.keys()` returns an array of all keys of the object.
2. `Object.values()` returns an array of all values of the object.
3. `Object.entries()` returns an array of key and value pairs.

```
let person = { name: "Rahul", age: 22, city: "Delhi" };
console.log(Object.keys(person));
// ["name", "age", "city"]
```

```
console.log(Object.values(person));  
// ["Rahul", 22, "Delhi"]  
console.log(Object.entries(person));  
// [["name","Rahul"], ["age",22], ["city","Delhi"]]
```

Q58. What is a Class in JavaScript?

A58. Class is a modern way to create objects in JavaScript introduced in ES6. It is a blueprint for creating multiple objects with same properties and methods. We use class keyword to create a class.

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  greet() {  
    console.log(`Hi, I am ${this.name} and I am ${this.age} years  
old.`);  
  }  
}  
let p1 = new Person("Rahul", 22);  
let p2 = new Person("Priya", 20);  
p1.greet(); // Hi, I am Rahul and I am 22 years old.  
p2.greet(); // Hi, I am Priya and I am 20 years old.
```

Q59. What is Inheritance in JavaScript?

A59. Inheritance means one class Inherits the properties and methods of another class. We use extends keyword for inheritance and use super keyword to call parent class constructor.

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  speak() {  
    console.log(this.name + " makes a sound.");  
  }  
}  
class Dog extends Animal {  
  bark() {  
    console.log(this.name + " says Woof!");  
  }  
}
```

```
let dog = new Dog("Bruno");
dog.speak(); // Bruno makes a sound.
dog.bark(); // Bruno says Woof!
```

Q60. What is the difference between map() and reduce()?

A60. Both are array methods but they work differently.

1. map() transforms every item and returns a new array with same length.
2. reduce() changes entire array into a single value like sum or total.

```
let numbers = [1, 2, 3, 4, 5];
// map – returns new array
let doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
// reduce – returns single value
let sum = numbers.reduce((total, n) => total + n, 0);
console.log(sum); // 15
```

Q61. What is the difference between let and const in loops?

A61. Both can be used in loops but they work differently.

1. let can be updated so it works perfectly in loops.
2. const cannot be updated so it cannot be used in normal for loops but can be used in for...of loops.

```
// let in loop – works fine
for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
// const in for...of – works fine
let fruits = ["Apple", "Banana", "Mango"];
for (const fruit of fruits) {
  console.log(fruit); // Apple, Banana, Mango
}
```

Q62. What is the difference between setTimeout and setInterval?

A62. Both are used to run code after a delay but they work differently.

1. setTimeout runs the code only once after a given time.
2. setInterval runs the code repeatedly after every given time.

```
// setTimeout – runs once after 2 seconds
setTimeout(() => {
  console.log("Runs only once after 2 seconds");
}, 2000);
// setInterval – runs every 1 second
let timer = setInterval(() => {
  console.log("Runs every 1 second");
}, 1000);
// Stop interval
clearInterval(timer);
```

Q63. What is the difference between innerHTML and textContent?

A63. Both are used to get or set content of an element but they work differently.

1. In innerHTML HTML tags are included with HTML content.
2. In textContent HTML tags are not included — it only gets or sets plain text.

```
let div = document.getElementById("myDiv");
// innerHTML – can set HTML tags
div.innerHTML = "<b>Hello World</b>"; // Shows bold text
// textContent – only plain text
div.textContent = "<b>Hello World</b>"; // Shows as plain text
```

Q64. What is the difference between == null and === null?

A64. Both check for null but they work differently.

1. == null returns true for both null and undefined.
2. === null returns true only for null — it is a strict check.

```
let a = null;
let b = undefined;
console.log(a == null); // true
console.log(b == null); // true – undefined also passes!
console.log(a === null); // true
console.log(b === null); // false – strict check
```

Q65. What is the difference between Array and Object in JavaScript?

A65. Both are used to store data but they work differently.

1. Array stores data in ordered list and values are accessed by index number.
2. Object stores data in key value pairs and values are accessed by key name.

```
// Array – ordered, accessed by index
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
// Object – key value pairs, accessed by key
let person = { name: "Rahul", age: 22 };
console.log(person.name); // Rahul
```

Q66. What is Callback Hell in JavaScript?

A66. Callback Hell means when we write callbacks inside callbacks multiple times — it makes code very messy and hard to read. It looks like a pyramid shape. So, we use Promises or `async await` to avoid callback hell.

```
// Callback Hell – messy code
loginUser("rahul", function(user) {
  getProfile(user.id, function(profile) {
    getOrders(profile.id, function(orders) {
      console.log(orders);
      // keeps going deeper and deeper...
    });
  });
});
// Solution – use async await
async function getData() {
  let user = await loginUser("rahul");
  let profile = await getProfile(user.id);
  let orders = await getOrders(profile.id);
  console.log(orders);
}
```

Q67. What is Prototype in JavaScript?

A67. Prototype is a hidden object that is attached with every object in JavaScript. It has properties and methods that other objects can use. For example ``map()`` and ``filter()`` methods come from ``Array.prototype``.

```
// Every array gets methods from Array.prototype
let arr = [1, 2, 3];
console.log(arr.hasOwnProperty("length")); // true
// Adding method to prototype
Array.prototype.sum = function() {
  return this.reduce((total, n) => total + n, 0);
};
```

```
let numbers = [1, 2, 3, 4, 5];  
console.log(numbers.sum()); // 15
```

Q68. What is the difference between `Object.freeze()` and `const`?

A68. Both are used to prevent changes but they work differently.

1. `const` means we cannot re-assign the variable but we can still change the object properties inside it.
2. `Object.freeze()` means we cannot change any property inside the object at all.

```
// const – variable cannot be re-assigned  
const person = { name: "Rahul", age: 22 };  
person.age = 25; // OK – property can be changed  
// person = {}; // ERROR – cannot re-assign  
// Object.freeze() – nothing can be changed  
const user = Object.freeze({ name: "Rahul", age: 22 });  
user.age = 25; // Silently ignored – no change  
console.log(user.age); // 22 – still same
```

Q69. What is Memoization in JavaScript?

A69. Memoization means storing the result of a function so it does not need to calculate the same thing again. If the same input comes again it returns the saved result. It makes code faster.

```
function memoize(fn) {  
  let cache = {};  
  return function(n) {  
    if (cache[n] !== undefined) {  
      console.log("From cache!");  
      return cache[n];  
    }  
    cache[n] = fn(n);  
    return cache[n];  
  };  
}  
  
function square(n) {  
  return n * n;  
}  
  
let fastSquare = memoize(square);  
console.log(fastSquare(5)); // Calculates: 25  
console.log(fastSquare(5)); // From cache! 25  
console.log(fastSquare(5)); // From cache! 25
```

Q70. What is Event Delegation in JavaScript?

A70. Event Delegation means we add one event listener on parent element instead of adding on every child element. It works because of event bubbling. It makes code faster and cleaner.

```
// Without Event Delegation – listener on every button (bad)
document.getElementById("btn1").addEventListener("click", handler);
document.getElementById("btn2").addEventListener("click", handler);
document.getElementById("btn3").addEventListener("click", handler);
// With Event Delegation – only one listener on parent (good)
document.getElementById("parent").addEventListener("click",
function(e) {
    if (e.target.tagName === "BUTTON") {
        console.log("Button clicked:", e.target.textContent);
    }
});
```
